Lab 2

Add an Existing Visual Studio Project to Source Control

Objective

In this lab, you will take an **existing C# Visual Studio project** that is currently stored only on your computer and place it under Git source control. You will then create a remote repository on GitHub and publish your project.

By the end of the lab, you will have practiced the following workflow:

```
Existing Visual Studio Project
            ↓
Initialize Git
            ↓
Make a Change
            ↓
Review Changes / Diff
            ↓
Commit
            ↓
Local Repository
            ↓
Publish / Push
            ↓
GitHub Remote Repository
```

Step 1 — Open the Existing Project

Open **Visual Studio 2022**.

Open an existing C# project using:

`File → Open → Project/Solution`

Select the appropriate `.sln` file.

Before continuing:

1. Build the project.
2. Run the application.
3. Confirm that it works correctly.

This gives you a working starting version before placing the project under source control.

Step 2 — Create a Local Git Repository

From the Visual Studio menu, select:

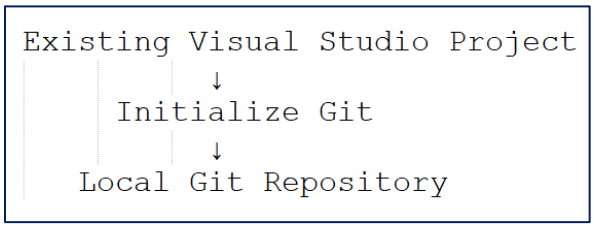
`Git → Create Git Repository`

Visual Studio will display the **Create a Git repository** window.

For this exercise, first create a **local Git repository** for the existing solution.

At this point, Git begins managing the project on your computer.

Conceptually:



Note: Creating a local Git repository does not automatically upload your project to GitHub.

Step 3 — Inspect the Project Files in Git

Open:

`View → Git Changes`

Visual Studio should display the files that Git has detected.

Typical project files may include:

`Program.cs`

`ProjectName.csproj`

`ProjectName.sln`

`.gitignore`

Visual Studio may automatically create a `.gitignore` file.

The `.gitignore` file prevents files and directories that normally should not be stored in source control from being included, such as:

`bin/`

```
obj/

.vs/
```

Take a moment to inspect the files that Git is tracking.

Step 4 — Make a Small Modification

Before creating your commit, make a small but meaningful modification to the existing application.

For example, suppose the application contains:

```
Console.WriteLine("Hello, World!");
```

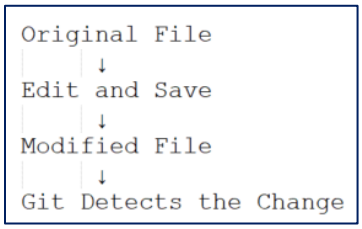
Change it to:

```
Console.WriteLine("Welcome to the Food Bank Inventory System!");
```

Save the file.

Git should now recognize that the source file has been modified.

Conceptually:



Step 5 — Review the Changes

Return to:

```
View → Git Changes
```

The modified file should appear in the **Git Changes** window.

Select the modified file to inspect its **diff**.

For example:

```
- Console.WriteLine("Hello, World!");
+ Console.WriteLine("Welcome to the Food Bank Inventory System!");
```

The `–` indicates content that was removed, while the `+` indicates content that was added.

Before committing, verify that:

- the expected files have changed;
- the modification is intentional;
- no unnecessary files are being included.

Tip: Do not blindly commit every changed file. Review your changes before creating a commit.

Step 6 — Create the First Commit

In the **Git Changes** window, enter a meaningful commit message.

For example:

```
Update application welcome message
```

Avoid vague messages such as:

```
changes
```

```
stuff
```

```
update
```

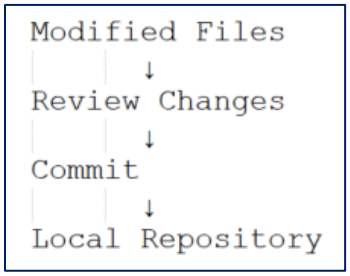
```
commit
```

Click:

```
Commit All
```

Your changes are now recorded in the **local Git repository**.

Conceptually:



At this point:

✓ Local repository contains the commit

✗ Project is not yet published to GitHub

Remember:

Commit records changes in the **local repository**.

Push sends committed changes to the **remote repository**.

Step 7 — Publish the Repository to GitHub

Now publish the local repository to GitHub.

In Visual Studio, look for:

`Publish to GitHub`

or:

`Push to Git service`

The exact wording may vary slightly depending on your Visual Studio 2022 version.

Choose **GitHub** as the remote hosting service.

Step 8 — Sign In to GitHub

If Visual Studio asks you to authenticate, sign in using your GitHub account.

Authorize Visual Studio or Git Credential Manager if requested.

Make sure you use the GitHub account under which you want the repository to be created.

Step 9 — Configure the GitHub Repository

Enter a repository name.

For example:

`FoodBankInventory`

You may also provide a short description.

Choose the appropriate repository visibility:

```
Public
```

or:

```
Private
```

For coursework or source code that should not be publicly available, select:

```
Private
```

Warning: If your source code is not intended for public access, make sure the repository is **Private** before publishing it.

Step 10 — Publish and Push the Project

Click the appropriate button, such as:

```
Create and Push
```

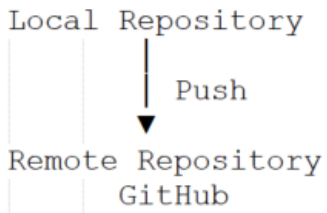
or:

```
Publish
```

Visual Studio should now:

1. create the remote repository on GitHub;
2. connect your local repository to the GitHub repository;
3. configure the remote, normally named origin;
4. push your committed project to GitHub.

Conceptually:



Step 11 — Verify the Remote Repository

You can verify the remote configuration in Visual Studio through the Git repository settings.

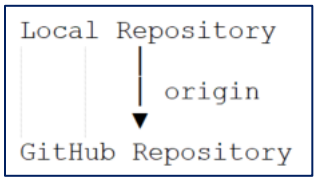
The remote will normally be named:

`origin`

and its address should look similar to:

`https://github.com/your-username/FoodBankInventory.git`

Conceptually:



Step 12 — Verify the Project on GitHub

Open GitHub in your browser and navigate to the repository.

You should now see the project files, for example:

`FoodBankInventory.sln`

`FoodBankInventory.csproj`

`Program.cs`

`.gitignore`

You should also see the commit that you created.

For example:

`Update application welcome message`

This confirms that the local commit was successfully pushed to the remote repository.

Step 13 — Verify Repository Visibility

On the GitHub repository page, check whether the repository is marked:

`Public`

or:

`Private`

Make sure the visibility matches your intention.

Warning: Never publish source code publicly if it contains information that should remain private.

Step 14 — Make Another Change

Now verify that the complete development workflow works.

Return to Visual Studio and make another small change to the application.

For example, add:

```
Console.WriteLine("Inventory application started
successfully.");
```

Save the file.

Step 15 — Review the New Change

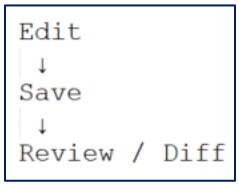
Open:

```
View → Git Changes
```

Confirm that the modified file appears.

Inspect the **diff** and make sure the change is correct.

The workflow is now:



Step 16 — Commit the New Change

Enter another meaningful commit message, for example:

```
Add application startup message
```

Click:

```
Commit All
```

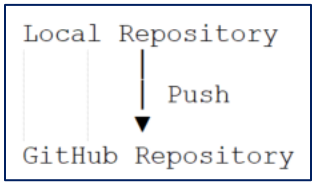
The new commit is now stored in your **local repository**.

Step 17 — Push the New Commit

In the **Git Changes** window, click:

Push

The new commit will be transferred from your local repository to GitHub.



Step 18 — Verify the New Commit on GitHub

Return to GitHub and refresh the repository page.

Confirm that:

- the updated source code is available;
- the new commit appears in the repository history;
- the commit message is correct.

You should now have at least two meaningful commits in your project history.

Complete Workflow

The complete process you performed in this lab is:

```
1. Open Existing Project
        ↓
2. Initialize Git
        ↓
3. Inspect Tracked Files
        ↓
4. Modify the Code
        ↓
5. Review Changes / Diff
        ↓
6. Commit
        ↓
   Local Repository
        ↓
7. Connect to GitHub
        ↓
8. Publish / Push
        ↓
   Remote Repository
        ↓
9. Make Another Change
        ↓
10. Review / Diff
        ↓
11. Commit
        ↓
12. Push
        ↓
13. Verify on GitHub
```

Key Concept to Remember

For normal development after the repository has been established, the workflow becomes:

```
Edit
 ↓
Review / Diff
 ↓
Add / Select Changes
 ↓
Commit
 ↓
Local Repository
 ↓
Push
 ↓
Remote Repository (GitHub)
```

Commit and Push are not the same operation.

- **Commit** records a version of your changes in your **local Git repository**.
- **Push** sends your local commits to the **remote repository on GitHub**.

This follows the source-control workflow from the course material, including the distinction between the working directory, local repository, and remote repository.